

Revisão do repositório Cash4You — todas as branches

Oct 2, 2026 · @Lucas Moraes

Resumo

O repositório tem 7 branches. Cinco já foram mergeadas na main e podem ser apagadas. A única com trabalho pendente é a devkbr, que não pode ser mergeada sem retrabalho.

A main está organizada e com uma boa modelagem de catálogo, mas ainda é um esqueleto: só há telas de listagem, nenhum teste e a venda ainda não existe. O comando de criação de grupos tem um bug: ele aponta para apps que não existem mais e não dá permissão nenhuma sobre produtos e serviços.

A devkbr tem a lógica de caixa mais madura do projeto, com 84 testes passando. Porém, ela saiu de uma versão antiga e usa uma estrutura incompatível com a main. Nas duas branches faltam autenticação e controle de permissão nas telas.

Panorama das branches

Só a devkbr tem commits que ainda não estão na main. As outras cinco já foram mergeadas e podem ser apagadas para limpar o repositório.

Branch	Autor	Último commit	Conteúdo	Situação
main	João	30/09	Base do projeto com todas as branches abaixo mergeadas	Branch principal
devkbr	Kleber	01/10	Caixa/PDV, agenda, cadastros completos, relatórios e 84 testes	Não mergeada: 2 commits à frente e 16 atrás da main
semana1	João	30/09	Arquivo de controle da entrega da semana 1	Idêntica à main; pode apagar
venda_item	João	30/09	App <code>vendaitem</code> e model <code>VendaItem</code>	Mergeada; pode apagar
refactor_catalogo	João	30/09	App <code>catalogo</code> com herança multi-table	Mergeada; pode apagar

Branch	Autor	Último commit	Conteúdo	Situação
dashboard	João	30/09	App dashboard com Bootstrap e herança de templates	Mergeada; pode apagar
usuario	João	28/09	Usuário customizado e comando <code>criar_grupos</code>	Mergeada; pode apagar

Para apagar uma branch remota já mergeada: `git push origin --delete nome-da-branch`.

Branch main

A main passa no `manage.py check` e não tem migrations pendentes, mas não tem nenhum teste. Funcionalmente, ela tem listagens de clientes, produtos e serviços, um dashboard com dois contadores e o model de usuário.

O que está bom

- O catálogo com herança multi-table (`Item` → `Produto` e `Servico`) é a melhor modelagem do projeto. Uma venda pode apontar para um único `Item`, seja produto ou serviço.
- `VendaItem` usa `on_delete=PROTECT`: um item já vendido não pode ser apagado do catálogo.
- O usuário customizado faz login por e-mail e tem um manager próprio.
- Os commits são pequenos e seguem o padrão `feat:` / `refactor:` / `docs:`. Cada funcionalidade teve sua branch e seu merge.

O que precisa corrigir

Prioridade	Problema	Onde	Sugestão
Alta	O comando <code>criar_grupos</code> busca permissões dos apps <code>produto</code> e <code>servico</code> , que não existem mais desde o refactor. Testado: Administrador fica com 8 permissões (só usuário e cliente), Gerente e Funcionário com 3 (só cliente). Ninguém recebe permissão sobre o catálogo.	<code>usuario/management/commands/criar_grupos.py</code>	Trocar para <code>catalogo</code> e incluir <code>vendaitem</code>
Alta	Nenhuma view exige login. No dashboard, o <code>@login_required</code> está comentado.	<code>dashboard/views.py</code> e demais views	Ativar <code>@login_required</code> e criar as telas de login e logout

Prioridade	Problema	Onde	Sugestão
Alta	<code>VendaItem</code> não tem venda: a FK para <code>Venda</code> está comentada e o model <code>Venda</code> não existe. Também não guarda a descrição do item no momento da venda.	<code>vendaitem/models.py</code>	Criar <code>Venda</code> (data, forma de pagamento, total, troco) e ligar os itens a ela
Média	Nenhum teste em nenhum app.	Todos os <code>tests.py</code>	Começar pelos models e pelo comando de grupos
Média	<code>Produto.quantidade</code> é <code>IntegerField</code> e aceita estoque negativo.	<code>catalogo/models.py</code>	<code>PositiveIntegerField</code>
Média	O CPF do cliente não é único nem validado, e o telefone aceita só 11 caracteres sem validação.	<code>cliente/models.py</code>	<code>unique=True</code> e validação de CPF (a devkbr já tem uma pronta)
Baixa	O dashboard mostra <code>total_vendas</code> fixo em 0. Dois itens do menu e os botões do cabeçalho levam para <code>#</code> .	<code>dashboard/views.py</code> , <code>templates/base.html</code>	Esperado nesta fase; resolver quando a venda existir
Baixa	Restos de cópia: a classe do admin de <code>VendaItem</code> se chama <code>ProdutoAdmin</code> , há <code>;</code> no fim de linhas do dashboard e um import sem uso (<code>default_urlconf</code>) em <code>core/urls.py</code> .	<code>vendaitem/admin.py</code> , <code>dashboard/views.py</code> , <code>core/urls.py</code>	Renomear e remover

Branch devkbr: o que está bom e deve ser mantido

Essas partes são o que mais vale aproveitar na adaptação para a estrutura da main.

- **Baixa de estoque segura.** A finalização usa um `update` condicional (`quantidade >= vendido`) dentro de `transaction.atomic()`. Duas vendas ao mesmo tempo não deixam o estoque negativo.
- **Edição e exclusão com trava.** As duas operações usam `select_for_update`, o que evita conflito entre operações simultâneas na mesma venda.
- **Histórico preservado.** O `ItemVenda` guarda descrição, preço unitário e subtotal do momento da venda. Se o cadastro mudar depois, a venda continua correta.
- **Dinheiro em `Decimal`.** Não há `float` em valores monetários, e o arredondamento da precificação usa `ROUND_HALF_UP`.
- **Testes.** São 84 testes, todos passando, e não há migrations pendentes.
- **Documentação.** O arquivo `docs/regras-de-negocio-e-funcionalidades.md` está claro e completo. Ele serve de referência para o time todo.

Branch devkbr: problema principal, a base desatualizada

A devkbr está 16 commits atrás da main. Ela foi criada antes de a main receber o refactor do catálogo, o app dashboard e o app vendaitem. Por isso, as duas branches têm estruturas diferentes para as mesmas coisas.

Parte do sistema	Como está na main	Como está na devkbr
Produto e serviço	App <code>catalogo</code> , com um model base <code>Item</code> (herança multi-table)	Apps separados <code>produto</code> e <code>servico</code>
Item de venda	App <code>vendaitem</code> : <code>VendaItem</code> com FK para <code>Item</code> e <code>on_delete=PROTECT</code>	App <code>venda</code> : <code>ItemVenda</code> com FKs separadas para produto e serviço e <code>on_delete=SET_NULL</code>
Dashboard	App <code>dashboard</code>	View dentro de <code>core</code>
Templates base	<code>templates/base.html</code>	<code>templates/base.html</code> reescrito, com versões antigas em <code>olds/</code>

Numa simulação de merge, aparecem conflitos em `core/settings.py`, `core/urls.py`, `templates/base.html`, `dashboard/urls.py` e nos models, views e templates de catálogo e serviço.

O conflito mais importante é no modelo de item de venda, porque não dá para manter os dois. A estrutura da main, com um `Item` unificado, deixa o caixa mais simples: os blocos `if tipo == produto / else serviço` em `venda/views.py` deixam de ser necessários.

Branch devkbr: outros pontos a corrigir

Os itens estão ordenados por prioridade. Os dois primeiros impedem o uso real do PDV.

Prioridade	Problema	Onde	Sugestão
Alta	Nenhuma view exige login. O model <code>Usuario</code> e os grupos (admin, gerente, funcionário) existem, mas não são usados. Qualquer pessoa na rede acessa o caixa e exclui vendas.	Todas as views	<code>@login_required</code> em tudo; <code>@permission_required</code> ou checagem de grupo para editar e excluir venda

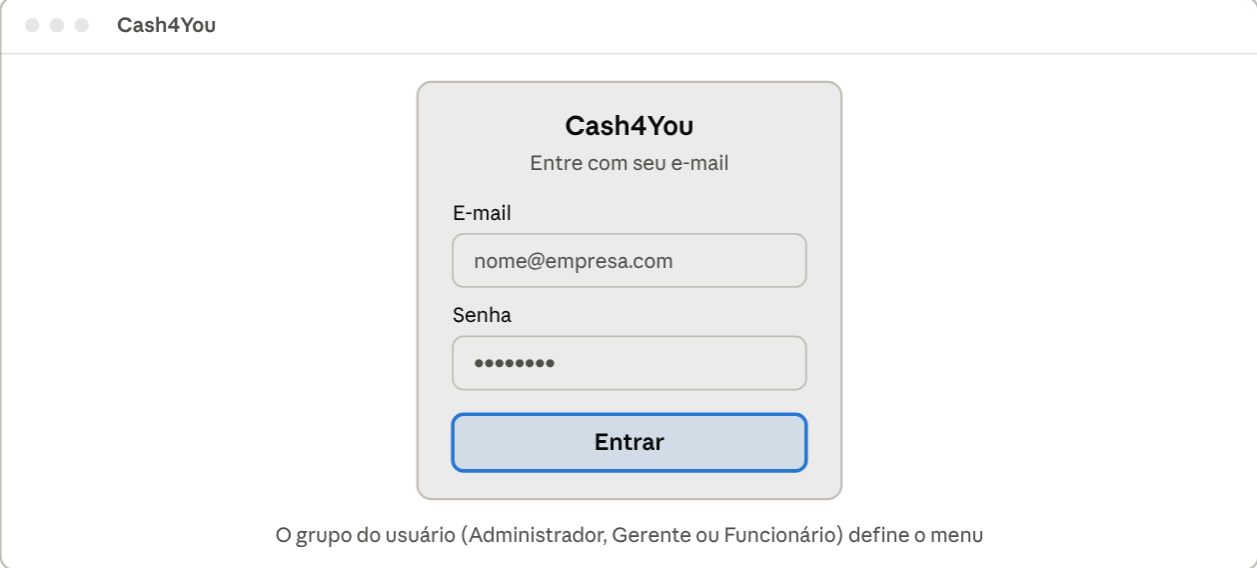
Prioridade	Problema	Onde	Sugestão
Alta	Excluir venda apaga o registro de vez, sem identificar quem fez nem por quê.	<code>venda/views.py</code> , <code>excluir_venda</code>	Trocar por cancelamento: campo de status, motivo, usuário e data; restringir a gerente e admin
Média	Editar venda recalcula itens e total com o preço atual do cadastro. Se o preço mudou, o valor histórico da venda muda junto.	<code>venda/views.py</code> , <code>editar_venda</code>	Manter o <code>preco_unitario</code> original dos itens já existentes
Média	<code>Servico.objects.get()</code> na finalização gera erro 500 se o serviço for excluído enquanto está no carrinho. O produto tem esse tratamento; o serviço não.	<code>venda/views.py</code> , <code>caixa</code> , ação finalizar	Tratar <code>DoesNotExist</code> e devolver mensagem de erro, como já é feito com produto
Média	Commits muito grandes: 2 commits com cerca de 5.700 linhas, incluindo "alterações em toda estrutura do projeto". Isso dificulta revisar e reverter.	Histórico do Git	Um commit por mudança lógica (model, view, template, testes)
Baixa	<code>requirements.txt</code> está em UTF-16, gerado pelo PowerShell. O pip instala normalmente, mas o Git trata o arquivo como binário e não mostra as diferenças. O problema também existe na main.	<code>requirements.txt</code>	Regerar em UTF-8
Baixa	Código morto e configuração local versionados.	<code>cliente/templates/olds/</code> , <code>.vscode/</code>	Remover e adicionar <code>.vscode/</code> ao <code>.gitignore</code>

Mockups das telas

Os mockups mostram como as telas ficariam depois das correções propostas, juntando a estrutura da main com as funcionalidades já feitas na devkbr. Nomes de produtos e valores são apenas ilustrativos.

Login

Login: nenhuma tela abre sem usuário autenticado



Cash4You

Cash4You
Entre com seu e-mail

E-mail
nome@empresa.com

Senha
••••••••

Entrar

O grupo do usuário (Administrador, Gerente ou Funcionário) define o menu

Tela de login · nova

Tela nova: hoje nenhuma das branches pede login. O acesso usa o **Usuario** por e-mail que já existe na main, e o grupo do usuário define o que aparece no menu.

Dashboard

Dashboard: indicadores do dia na primeira tela



Dashboard · 4 indicadores, 2 gráficos, alerta de estoque

Junta o app `dashboard` da main com os gráficos da devkbr. O contador de vendas deixa de ser fixo em 0 quando o model `Venda` existir, e o alerta de reposição usa o limite de 5 unidades da devkbr.

Caixa (PDV)

Caixa: carrinho à esquerda, pagamento à direita

Cash4You

Dashboard

Clientes

Catálogo

Caixa

Vendas

Agenda

Relatórios

Funcionário · Sair

Caixa

Buscar produto ou serviço

Adicionar

Item	Qtd	Preço	Subtotal
Produto A	2	R\$ 45,00	R\$ 90,00
Serviço A	1	R\$ 60,00	R\$ 60,00
Produto B	3	R\$ 12,50	R\$ 37,50

Quantidade limitada ao estoque disponível

Total

R\$ 187,50

Forma de pagamento

Dinheiro

Pix

Crédito

Débito

Valor recebido

R\$ 200,00

Troco

R\$ 12,50

WhatsApp (opcional)

(35) 9 0000-0000

Finalizar venda

Caixa / PDV · carrinho e pagamento

Mantém o fluxo da devkbr: carrinho com estoque validado, quatro formas de pagamento, troco calculado no dinheiro e envio opcional do resumo pelo WhatsApp. A diferença é que produtos e serviços vêm do mesmo `catalogo.Item`, numa busca só.

Histórico de vendas

Vendas: cancelamento registrado no lugar da exclusão

Cash4You

Dashboard

Cientes

Catálogo

Caixa

Vendas

Agenda

Relatórios

Gerente · Sair

Histórico de vendas

Filtrar por período

Nº	Data e hora	Pagamento	Total	Status	Ações
#128	02/10 14:32	Dinheiro	R\$ 187,50	Concluída	Detalhes · Cancelar
#127	02/10 11:05	Pix	R\$ 60,00	Concluída	Detalhes · Cancelar
#126	01/10 17:48	Crédito	R\$ 245,00	Cancelada	Detalhes
	Motivo: cliente desistiu · por Gerente em 01/10 18:02				
#125	01/10 09:20	Débito	R\$ 32,00	Concluída	Detalhes · Cancelar

Cancelar pede motivo, devolve o estoque e só aparece para Gerente e Administrador

Histórico de vendas · com cancelamento

A venda cancelada continua na lista, com motivo, usuário e horário, em vez de ser apagada. O botão Cancelar só aparece para Gerente e Administrador.

Catálogo de produtos

Catálogo: lucro e estoque calculados por produto

Cash4You

Dashboard

Cientes

Catálogo

Caixa

Vendas

Agenda

Relatórios

Gerente · Sair

Catálogo · Produtos

Novo produto

Buscar por descrição

Ordenar: A-Z

Descrição	Custo	Preço	Estoque	Lucro unit.	Ações
Produto A	R\$ 20,00	R\$ 45,00	12	R\$ 21,40	Editar
Produto B	R\$ 5,00	R\$ 12,50	3	R\$ 6,50	Editar
Produto C	R\$ 30,00	R\$ 70,00	8	R\$ 34,40	Editar
Totais			23		

Valor do estoque: R\$ 1.137,50 · Lucro potencial do estoque: R\$ 551,50

Estoque abaixo de 5 unidades em destaque

Catálogo de produtos · listagem com lucro e totais

Usa o app `catalogo` da main com as colunas de lucro da devkbr. No exemplo, o lucro unitário considera 8% de despesas variáveis (preço – custo – 8% do preço). A aba de serviços segue o mesmo modelo, sem a coluna de estoque.

Agenda

Agenda: blocos de 30 minutos, ocupados ficam riscados

Cash4You

Dashboard
Clientes
Catálogo
Caixa
Vendas
Agenda
Relatórios

Funcionário · Sair

Novo agendamento

Data
02/10/2026

Horários (selecione um ou mais)

07:00	07:30	08:00	08:30
09:00	09:30	10:00	10:30
11:00	11:30	12:00	12:30

Recorte até 12:30; o dia vai das 07:00 às 23:00 (32 blocos)

☒ Selecionado ☐ Ocupado ☐ Livre

Nome
Nome do cliente

WhatsApp
(35) 9 0000-0000

Serviços (opcional)
✓ Serviço A
Serviço B

Agendar 09:30-10:30

Agenda · seleção de blocos de 30 minutos

Segue as regras da devkbr: blocos de 30 minutos das 07:00 às 23:00, horários ocupados riscados e bloqueados, serviços opcionais. Essa tela ainda não existe na main, então o app `agenda` precisa ser trazido junto no rebase.

Próximos passos sugeridos

O caminho mais curto é reaproveitar a lógica e os testes da devkbr em cima da estrutura atual da main.

1. Alinhar com o time a modelagem da venda antes de mexer no código. A sugestão é usar `catalogo.Item` + `vendaitem` da main, adicionando nela o que falta: model `Venda`, forma de pagamento, valor recebido, troco e os campos de retrato do item (descrição, preço unitário e subtotal).
2. Atualizar a branch com a main (`git fetch` + `git rebase origin/main`, ou criar uma branch nova a partir da main).

3. Portar o caixa para o novo modelo, mantendo a baixa de estoque com update condicional e `transaction.atomic()`.
4. Adaptar os 84 testes para a nova estrutura e garantir que todos passem.
5. Adicionar login obrigatório e permissões por grupo.
6. Trocar exclusão de venda por cancelamento com registro de usuário e motivo.
7. Corrigir na main o comando `criar_grupos`, apontando para `catalogo` e `vendaitem`, e os demais itens das tabelas acima.
8. Abrir um pull request para a main com commits pequenos, para facilitar a revisão. Depois do merge, apagar as branches que já estão na main.